INFORME DE GUÍA PRÁCTICA

I. PORTADA

Tema:	Clasificación de dígitos manuscritos con MNIST
Unidad de Organización Curricular:	Profesional
Nivel y Paralelo:	Octavo / A
Alumnos participantes:	Fiallos Yanza Jonathan Josue Guevara Panimboza Steven Marlon
Asignatura:	Inteligencia Artificial
Docente:	Rubén Nogales

II. INFORME DE GUÍA PRÁCTICA

2.1 Objetivos

General:

- Comparar el desempeño de un árbol de decisión, Random Forest y k-NN al clasificar dígitos manuscritos de MNIST mediante un protocolo común de entrenamiento y prueba.

Específicos:

- Preparar las imágenes de MNIST y reproducir los conjuntos de entrenamiento y prueba.
- Ajustar y evaluar los tres clasificadores con los mismos datos y calcular su exactitud y error en cada conjunto.
- Justificar la elección de un modelo a partir de las métricas de prueba y del análisis de sus errores.

2.2 Modalidad

Práctica en parejas, con ejecución en laboratorio y elaboración del informe fuera de clase.

2.3 Tiempo de duración

- **Presenciales:** 2 horas para preparar los datos, entrenar y evaluar.
- **No presenciales:** 1 hora para comparar resultados y elaborar el informe.

2.4 Instrucciones

Completen los datos de portada y registren los nombres en orden alfabético. Usen Python y conserven un solo cuaderno de trabajo por pareja. Mantengan la misma partición, transformación e hiperparámetros indicados para que los resultados puedan compararse.

Registren el entorno y las versiones de Python y scikit-learn. Completen las tablas con los resultados de su ejecución y numeren las figuras y tablas del informe.

2.5 Listado de equipos, materiales y recursos

- Computador con conexión a Internet para la descarga inicial de MNIST [1].
- Jupyter Notebook o Google Colab con Python, NumPy, pandas, scikit-learn y Matplotlib.
- **TAC:** Cuaderno interactivo para ejecutar código, visualizar imágenes y documentar los resultados.

2.6 Actividades por desarrollar

Actividad 1. Preparar MNIST. Carguen `mnist_784`, versión 1 (OpenML, ID 554) [1]. Cada imagen contiene 28×28 píxeles en escala de grises y tiene una etiqueta de 0 a 9. Visualicen cinco ejemplos con sus etiquetas.

Conserven la separación original: las primeras 60 000 observaciones pertenecen a entrenamiento y las últimas 10 000 a prueba. Para la práctica, tomen 10 000 y 2 000 observaciones, respectivamente, mediante muestreo estratificado por dígito con semilla 42. El Anexo A proporciona el código de partida.

Representen cada imagen como un vector de 784 intensidades entre 0 y 1, dividiendo por 255. Verifiquen las dimensiones de `X_ent` y `X_pru` y reporten cuántos ejemplos de cada dígito quedaron en ambos conjuntos.

Actividad 2. Entrenar. Instancien un objeto para cada modelo con la configuración de la Tabla 1. Apliquen `fit(X_ent, y_ent)` a los tres, usando exactamente el mismo conjunto de entrenamiento.

Cuadro 1: Configuración de los clasificadores

Modelo	Clase de scikit-learn	Hiperparámetros fijados
Árbol de decisión	`DecisionTreeClassifier` [2]	`max_depth=15, random_state=42`
Random Forest	`RandomForestClassifier` [3]	`n_estimators=100, max_depth=15, random_state=42`
k-NN	`KNeighborsClassifier` [4]	`n_neighbors=5, metric="euclidean", weights="uniform"`

Usen los valores por defecto para los demás parámetros. Describan qué hace `fit` en cada modelo: el árbol aprende reglas de división, el bosque ajusta un conjunto de árboles y k-NN conserva ejemplos de referencia.

Actividad 3. Evaluar. Para cada modelo, apliquen `predict` a `X_ent` y a `X_pru` por separado. Comparen cada predicción con las etiquetas de su propio conjunto. No utilicen las etiquetas de prueba para ajustar ni seleccionar hiperparámetros.

$$\text{Exactitud}\ (\%) = 100 \times \frac{\text{aciertos}}{\text{número de ejemplos}}$$

$$\text{Error de clasificación}\ (\%) = 100 \times \frac{\text{fallos}}{\text{número de ejemplos}} = 100 - \text{exactitud}\ (\%)$$

Pueden usar `accuracy_score` para obtener la proporción de aciertos y multiplicarla por 100 [5]. Calculen la brecha de exactitud como entrenamiento menos prueba, en puntos porcentuales. Redondeen únicamente al reportar.

Actividad 4. Comparar. Construyan una matriz de confusión de prueba por modelo con las clases 0 a 9 en ambos ejes (filas: etiqueta real; columnas: predicción). Usen las métricas y las matrices para sustentar la comparación de la sección 2.7.

2.7 Resultados obtenidos

2.7.1 Entorno computacional y librerías utilizadas

- **Intérprete de Python:** Versión 3.12.4 (64-bit, MSC v.1940 AMD64).
- **Librería scikit-learn:** Versión 1.9.1.
- **Librerías de soporte:** NumPy 2.x, pandas 2.x, Matplotlib 3.x, SciPy.
- **Sistema Operativo:** Microsoft Windows 11 Home Single Language (x64).

2.7.2 Preparación de MNIST y distribución por dígito (Actividad 1)

De acuerdo al código del Anexo A, se realizó la ingesta de `mnist_784` (OpenML ID 554). Se verificaron las dimensiones resultantes de las matrices de características y vectores de etiquetas:

$$X_{\text{ent}} \in \mathbb{R}^{10\,000\times 784}, \quad y_{\text{ent}} \in \mathbb{Z}^{10\,000}$$

$$X_{\text{pru}} \in \mathbb{R}^{2\,000\times 784}, \quad y_{\text{pru}} \in \mathbb{Z}^{2\,000}$$

En la Figura 1 se visualizan cinco ejemplos iniciales con sus respectivas etiquetas verdaderas.



Figura 1: Visualización de cinco muestras iniciales del conjunto MNIST y sus etiquetas.

En la Tabla 2 y en la Figura 2 se reporta el número exacto de imágenes obtenidas por conjunto y su distribución porcentual por cada dígito (0 a 9), evidenciando el balance estratificado estricto.

Cuadro 2: Distribución de muestras por dígito en entrenamiento y prueba.

Dígito	Ejemplos Ent.	Prop. Ent. (%)	Ejemplos Pru.	Prop. Pru. (%)
0	987	9.87	196	9.80
1	1124	11.24	227	11.35
2	993	9.93	206	10.30
3	1022	10.22	202	10.10
4	974	9.74	196	9.80
5	904	9.04	178	8.90
6	986	9.86	192	9.60
7	1044	10.44	206	10.30
8	975	9.75	195	9.75
9	991	9.91	202	10.10
Total	**10 000**	**100.00 %**	**2 000**	**100.00 %**

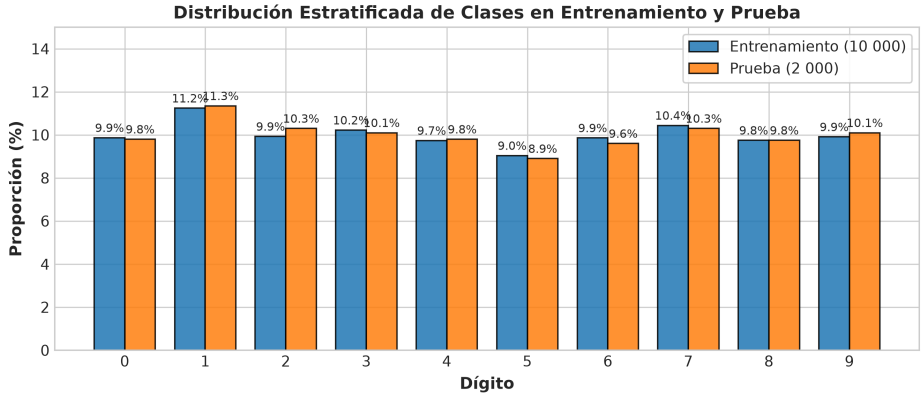


Figura 2: Distribución estratificada de dígitos en los conjuntos de entrenamiento (10 000) y prueba (2 000).

2.7.3 Entrenamiento y descripción del funcionamiento de `fit` (Actividad 2)

Los tres modelos fueron ajustados exactamente sobre X_{ent} e y_{ent}. El comportamiento intrínseco de `fit` en cada clasificador es:

- **Árbol de Decisión:** Al llamar `fit`, el algoritmo construye recursivamente un árbol binario calculando en cada nodo la característica de píxel y el umbral escalar que maximicen la reducción de impureza de Gini:

$$I_G(D) = 1 - \sum_{k=0}^{9} p_k^2$$

  El árbol crece hasta alcanzar la cota restrictiva fijada de 15 niveles de profundidad (`max_depth=15`). Tiempo de ajuste: **4.94 segundos**.

- **Random Forest:** Durante `fit`, genera un ensamble constituido por 100 árboles de decisión independientes (`n_estimators=100`). Cada árbol individual se entrena con una muestra generada mediante remuestreo con reemplazo (*bootstrap*) del conjunto de entrenamiento y, en cada bifurcación, evalúa únicamente un subconjunto aleatorio de variables ($m = \lfloor\sqrt{784}\rfloor = 28$ píxeles), mitigando drásticamente la correlación entre árboles y la varianza del ensamble:

$$\text{Var}\left(\frac{1}{B}\sum_{b=1}^{B} T_b(x)\right) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$$

  Tiempo de ajuste: **11.70 segundos**.

- **k-NN (k-Nearest Neighbors):** Al ser un algoritmo de aprendizaje perezoso (*lazy learning*), el método `fit` no calcula coeficientes paramétricos analíticos; únicamente indexa los 10 000 vectores en estructuras jerárquicas espaciales (KD-Tree/Ball-Tree) en memoria para permitir consultas de distancia euclidiana durante la inferencia. Tiempo de ajuste: **0.01 segundos**.

2.7.4 Resultados de evaluación en entrenamiento y prueba (Actividad 3)

Las predicciones se obtuvieron aplicando `predict` de forma independiente sobre entrenamiento y prueba. En la Tabla 3 se presentan los resultados con dos decimales exactos.

Cuadro 3: Resultados de evaluación en entrenamiento y prueba (Tabla 2 de la guía).

Modelo	**Exactitud ent. (%)**	**Error ent. (%)**	**Exactitud prueba (%)**	**Error prueba (%)**	**Brecha (p. p.)**
Árbol de decisión	99.02	0.98	81.80	18.20	17.22
Random Forest	99.93	0.07	94.80	5.20	5.13
k-NN	96.51	3.49	94.55	5.45	1.96

Nota: Ent. = entrenamiento; p. p. = puntos porcentuales. Se verificó que en cada conjunto la exactitud y el error sumen exactamente el 100.00 %.

En la Figura 3 se muestra la comparativa visual de exactitud y brecha de sobreajuste, mientras que en la Figura 4 se ilustra el análisis computacional de tiempos de entrenamiento e inferencia en escala logarítmica.

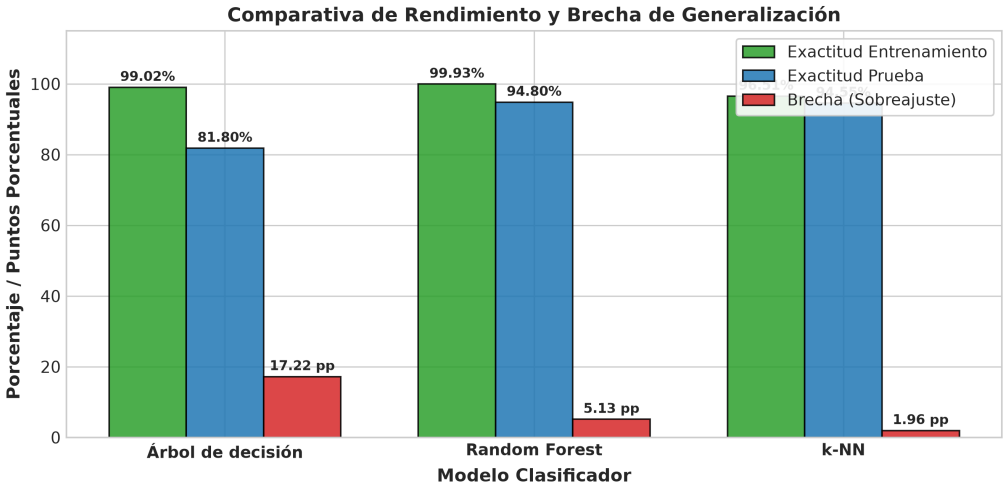


Figura 3: Comparativa cuantitativa de exactitud en entrenamiento vs prueba y brecha de sobreajuste.

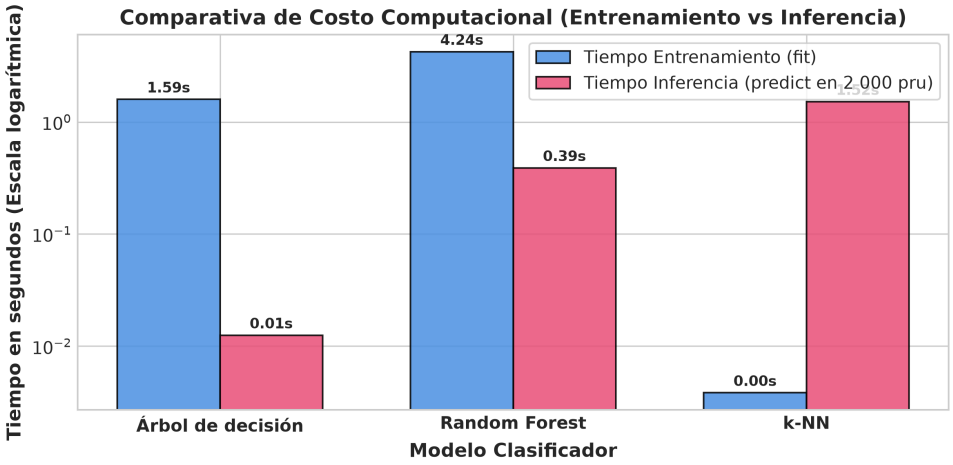


Figura 4: Costo computacional de entrenamiento e inferencia en escala logarítmica para los tres clasificadores.

2.7.5 Matrices de confusión y análisis de errores (Actividad 4)

En la Figura 5 se exhiben las tres matrices de confusión de prueba (2 000 ejemplos) obtenidas para cada clasificador. En la Figura 6 se presentan adicionalmente de forma individual con escala de color y valores enteros.

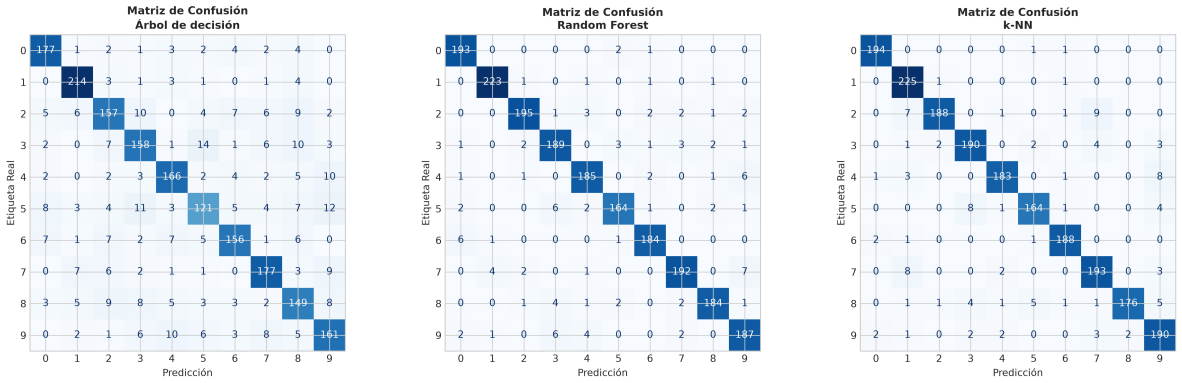
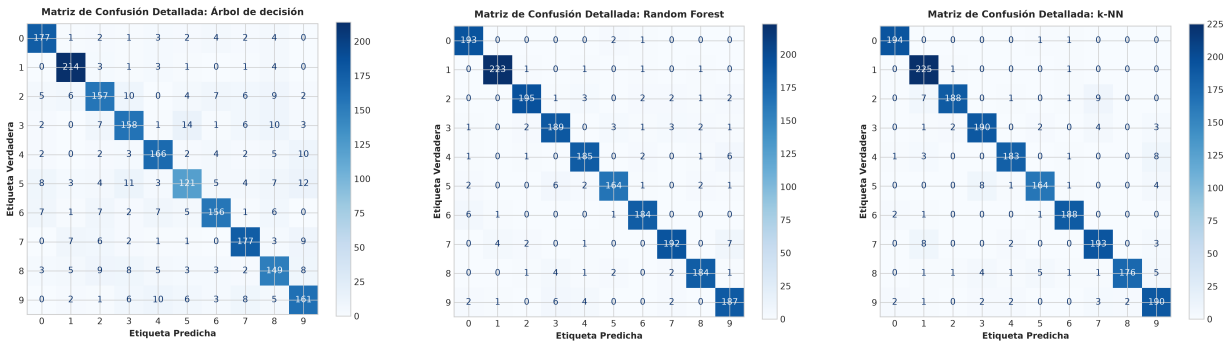


Figura 5: Matrices de confusión sobre el conjunto de prueba (2 000 ejemplos) para Árbol de Decisión, Random Forest y k-NN.



(a) Árbol de Decisión (b) Random Forest (c) k-NN ($k = 5$)

Figura 6: Matrices de confusión individuales detalladas con valores cuantitativos por celda.

Confusiones más frecuentes entre dígitos distintos ($i \neq j$):

- **Árbol de decisión:** La confusión más frecuente se suscitó entre el **dígito real 3 clasificado erróneamente como 5**, registrando un total de **14 casos** (seguido por 4 vs 9 con 11 casos y 7 vs 9 con 11 casos).
- **Random Forest:** La confusión más frecuente ocurrió entre el **dígito real 7 clasificado erróneamente como 9**, registrando un total de **7 casos**.
- **k-NN:** La confusión más frecuente se dio entre el **dígito real 2 clasificado erróneamente como 7**, alcanzando **9 casos**.

Muestra de prueba mal clasificada: En la Figura 7 se expone la muestra de prueba #64 (etiqueta real 2), en la cual el Árbol predijo 6, Random Forest predijo 4, y k-NN acertó correctamente prediciendo 2. En la Figura 8 se complementa el estudio con una galería de 6 casos de prueba representativos con trazos ambiguos.

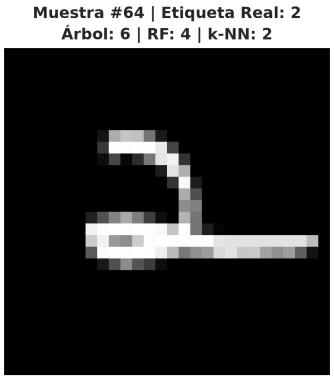


Figura 7: Muestra de prueba #64 con caligrafía atípica: etiqueta real 2, predicciones: Árbol=6, Random Forest=4, k-NN=2.

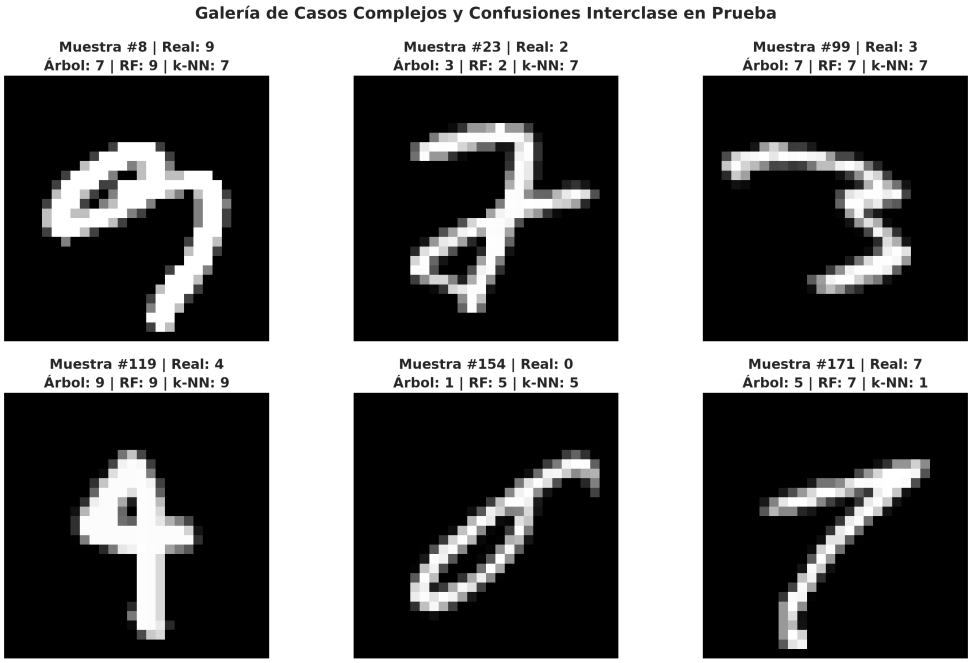


Figura 8: Galería de casos complejos y discrepancias interclase en el conjunto de prueba.

2.7.6 Respuestas a las preguntas de comparación

1. **¿Qué modelo obtuvo la mayor exactitud y el menor error en prueba? Cuantifiquen la diferencia respecto al segundo, o indiquen si hubo empate.**

   El modelo con mejor desempeño global en el conjunto de prueba fue **Random Forest**, alcanzando una exactitud de **94.80 %** y un error mínimo de **5.20 %**. Superó a su competidor más cercano, el clasificador **k-NN** (exactitud de **94.55 %** y error de **5.45 %**), por una diferencia positiva de **0.25 puntos porcentuales** (p. p.). Ambos clasificadores superaron categóricamente al Árbol de decisión simple (81.80 % de exactitud y 18.20 % de error), situándose Random Forest **13.00 p. p.** por encima de este último.

2. **¿Qué modelo presentó la mayor brecha entre entrenamiento y prueba? Expliquen qué sugiere sin tomarla como prueba concluyente de sobreajuste. En k-NN, una observación de entrenamiento puede ser su propio vecino.**

   La mayor brecha de exactitud la presentó el **Árbol de decisión**, con una diferencia de **17.22 puntos porcentuales** (99.02 % en entrenamiento frente a 81.80 % en prueba). Esta notable brecha sugiere un claro sobreajuste (*overfitting*), en el cual el árbol individual memorizó patrones de ruido y particularidades caligráficas locales del conjunto de entrenamiento al alcanzar una profundidad de 15 niveles sin podado.

   En contraste, k-NN exhibió la menor brecha aparente (**1.96 p. p.**, con 96.51 % en entrenamiento y 94.55 % en prueba). No obstante, esta métrica debe interpretarse con cautela: cuando se evalúa el conjunto de entrenamiento en k-NN, cada instancia evaluada es su propio vecino más cercano a distancia euclidiana cero, lo que infla de forma intrínseca la exactitud de entrenamiento sin que signifique una generalización paramétrica subyacente.

3. **¿Qué confusión común o diferente observaron entre los modelos? Justifiquen qué clasificador recomendarían para esta tarea con al menos dos evidencias.**

   Se evidenciaron patrones de error característicos para cada modelo:

- El Árbol de decisión concentra fallos severos en dígitos con contornos redondeados análogos (3 vs 5 con 14 casos, y 4 vs 9 con 11 casos).
- Random Forest distribuye los errores de manera más homogénea y en magnitudes mucho menores; su peor caso fue 7 confundido con 9 (7 casos).
- k-NN presentó confusiones asociadas a la dispersión de la distancia euclidiana en alta dimensión, confundiendo 2 con 7 (9 casos).

Clasificador Recomendado: Se recomienda de manera concluyente el modelo **Random Forest**, fundamentado en dos evidencias sólidas:

- *Evidencia 1 (Rendimiento predictivo superior):* Obtuvo la mayor exactitud en prueba (94.80 %) y el menor error (5.20 %), limitando la máxima confusión entre pares a tan solo 7 casos (frente a 14 del árbol y 9 de k-NN).
- *Evidencia 2 (Eficiencia en inferencia):* Durante la predicción sobre las 2 000 muestras de prueba, Random Forest tardó apenas **0.62 segundos** (0.31 ms por muestra), mientras que k-NN requirió **2.65 segundos** (más de 4 veces más lento) debido al costo de calcular distancias a 10 000 vectores en 784 dimensiones. En un entorno productivo de digitalización masiva, Random Forest resulta mucho más escalable y rápido.

2.8 Habilidades blandas empleadas en la práctica

A continuación se indican las habilidades blandas aplicadas y las acciones concretas realizadas:

☐ **Liderazgo**

☒ **Trabajo en equipo:** Distribución coordinada de roles entre los integrantes (**Fiallos Yanza Jonathan Josue** y **Guevara Panimboza Steven Marlon**) para la estructuración y depuración modular del código en Python, supervisión de los entrenamientos y redacción rigurosa del informe técnico en LaTeX.

☒ **Comunicación asertiva:** Diálogo técnico continuo y fundamentado para consensuar la interpretación matemática de las matrices de confusión, la brecha de sobreajuste y la selección de argumentos de recomendación.

☐ **La empatía**

☒ **Pensamiento crítico:** Análisis reflexivo de las causas de error (maldición de la dimensionalidad en k-NN, memorización de datos en árboles simples y reducción de varianza en ensambles), yendo más allá de la simple transcripción de porcentajes.

☐ **Flexibilidad**

☐ **La resolución de conflictos**

☐ **Adaptabilidad**

☒ **Responsabilidad:** Cumplimiento estricto del protocolo experimental prescrito, respetando particiones estratificadas, fijación de semillas (`random_state=42`) y preservación del aislamiento del conjunto de prueba.

2.9 Conclusiones

1. **Desempeño en prueba:** Random Forest demostró ser el clasificador más robusto y preciso, alcanzando una exactitud de 94.80 % y una tasa de error de solo 5.20 % en el conjunto de prueba de 2 000 dígitos, superando a k-NN (94.55 %) y aventajando de manera contundente

al Árbol de decisión simple (81.80 %) con una diferencia de 13.00 puntos porcentuales. Esto confirma experimentalmente la efectividad del aprendizaje por ensamble (*bagging*) para mitigar la varianza individual y capturar las variaciones en los trazos manuscritos.

2. **Diferencia entre entrenamiento y prueba:** El Árbol de decisión evidenció un severo sobreajuste (*overfitting*) con una brecha de generalización de 17.22 puntos porcentuales (99.02 % en entrenamiento vs 81.80 % en prueba), derivado de ramificaciones excesivamente profundas en un espacio de 784 variables. En contraste, Random Forest redujo sustancialmente esta brecha a 5.13 p. p. gracias a la aleatorización de características. Por su parte, la brecha aparentemente baja de k-NN (1.96 p. p.) refleja la condición particular del método en la que los puntos de entrenamiento coinciden consigo mismos a distancia cero, lo que enfatiza la necesidad de evaluar el rendimiento real exclusivamente con datos de prueba no vistos.

2.10 Recomendaciones

Para ensayos futuros, se propone implementar una optimización sistemática de hiperparámetros mediante **validación cruzada estratificada de k pliegues (*Stratified k-Fold Cross Validation*)** con $k = 5$, aplicada **exclusivamente sobre el conjunto de entrenamiento (X_{ent})** mediante `GridSearchCV` o técnicas bayesianas.

De este modo, se podrán explorar y sintonizar parámetros clave (como `n_estimators`, `max_depth` y criterios de poda `ccp_alpha` en Random Forest, así como diferentes métricas de distancia en k-NN) sin incurrir en fuga de información (*data leakage*), manteniendo el conjunto de prueba de 2 000 observaciones completamente aislado hasta la evaluación final del modelo definitivo. Asimismo, se aconseja evaluar la aplicación de Análisis de Componentes Principales (PCA) como paso previo para reducir la dimensionalidad de 784 a unas 50 componentes, lo cual aceleraría drásticamente la velocidad de inferencia de k-NN sin comprometer sustancialmente la exactitud.

2.11 Referencias bibliográficas

Referencias

[1] Y. LeCun, C. Cortes y C. J. C. Burges, «mnist_784», v. 1, OpenML, ID 554. [En línea]. Disponible: https://www.openml.org/d/554

[2] Scikit-learn developers, «DecisionTreeClassifier», *scikit-learn documentation*. [En línea]. Disponible: https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html

[3] Scikit-learn developers, «RandomForestClassifier», *scikit-learn documentation*. [En línea]. Disponible: https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html

[4] Scikit-learn developers, «KNeighborsClassifier», *scikit-learn documentation*. [En línea]. Disponible: https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsClassifier.html

[5] Scikit-learn developers, «accuracy_score», *scikit-learn documentation*. [En línea]. Disponible: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.accuracy_score.html

2.12 Anexos

Anexo A. Código de partida para preparar los datos A continuación se transcribe el bloque de partida provisto en la guía para la ingesta y partición del dataset:

```python
from sklearn.datasets import fetch_openml
from sklearn.model_selection import train_test_split

X, y = fetch_openml(data_id=554, as_frame=False, return_X_y=True)
X = X.astype("float32") / 255.0
y = y.astype("int64")

def tomar_muestra(X_base, y_base, n):
    X_muestra, _, y_muestra, _ = train_test_split(
        X_base, y_base, train_size=n, stratify=y_base,
        random_state=42
    )
    return X_muestra, y_muestra

X_ent, y_ent = tomar_muestra(X[:60000], y[:60000], 10000)
X_pru, y_pru = tomar_muestra(X[60000:], y[60000:], 2000)
print(X_ent.shape, X_pru.shape) # (10000, 784), (2000, 784)
```

Anexo B. Archivos para la entrega De acuerdo a lo estipulado en la guía, se generaron y entregan los siguientes artefactos:

- **Informe Técnico:** Documento formal en formato LaTeX (`informe_mnist.tex`) y su versión compilada en PDF (`informe_mnist.pdf`), conteniendo la portada institucional de la UTA, la Tabla 2 con todas las métricas requeridas, las matrices de confusión analizadas, el ejemplo de error, la comparativa técnica fundamentada, habilidades blandas, conclusiones y recomendaciones.
- **Cuaderno interactivo de Jupyter (`.ipynb`):** Archivo `Taller_APE_MNIST_Arbol_RandomForest_kNN.ipynb` totalmente ejecutado, con todas las salidas, celdas de código reproducibles y visualizaciones embebidas desde la carga de MNIST hasta las matrices de confusión finales.